

Title:

Scaling the Swiss National ABM:

A High-Performance, Native Java Implementation for generating daily plans in MATSim

Authors:

Joschka Bischoff, Annette Knupp (Swiss Federal Railways SBB)

Abstract:

As Activity-Based Models (ABMs) grow in complexity and scale, coupling them with agent-based traffic simulations like MATSim frequently creates severe computational bottlenecks. Traditionally, the SBB's national-scale SIMBA MOBi demand model relied on a heavily vectorized Python and PTV Visum pipeline to generate endogenous daily plans for the nine million Swiss agents. Due to high memory consumption, slow I/O, and the inherent inefficiencies of bridging disparate software ecosystems, this pre-simulation pipeline required over 16 hours of execution time.

This presentation details the complete architectural re-engineering of the SIMBA MOBi plan generation pipeline into a pure, dependency-free, MATSim-native Java engine. We successfully migrated the entire behavioral core—including Tour Building, Primary and Secondary Location Choice, Mode Choice, Start Time Choice, and Facility Assignment—directly into the MATSim execution framework.

We highlight several key software engineering and algorithmic innovations required to achieve this scale without exhausting JVM heap space:

1. **High-Performance Discrete Choice Engines:** The implementation of generalized, multi-threaded Logit and Linear Probability selectors, utilizing `ThreadLocal`-cached AST parsers to evaluate millions of complex utility functions in milliseconds.
2. **Memory-Optimized Impedance Caching:** Replacing nested object maps with flattened, primitive `float[][]` caches for Skim, Kalif, and Logsum matrices, eliminating over 100 GB of Java object overhead and preventing Garbage Collection starvation.
3. **Strict Chronological "Calendar Tetris":** Replacing Python's heuristic arrays with a 100-iteration Simulated Annealing combinatorial search. This algorithm enforces strict MATSim topological realities (e.g., 1-way vs. 2-way detour rubberbanding, non-overlapping activities, and mandatory home-activity bridging) while perfectly hitting calibrated macroscopic time-of-day distributions.

The resulting native Java pipeline reduced the execution time of the 9.3-million-agent scenario from 16 hours to around 90 minutes. This >10x speedup allows us to evaluate further application cases, including generating plans for multi-day simulations. The model framework, including a set of sample parameters, is open source and will be published in time for the MATSim User Meeting.